INTERNSHIP PROJECT REPORT

ON

LINUX PRIVILEGE ESCALATION AUTOMATION TOOLKIT

Submitted as part of the **Cyber Security Internship** Program

Internship Period

December 2025 – June 2026

Supervised By:

Mr. Harshit

Submitted By:

Ravi

Organization:

Unified Mentor

INDEX

Sr. No.	Title	Page No.
1.	Abstract	4
2.	Introduction	5-6
3.	Problem Statement	7
4.	Project Objective	8
5.	Scope of the Project • 5.1 In Scope • 5.2 Out of Scope	9
6.	Tools & Technologies Used	10
7.	Methodology / Workflow	11-13
8.	Implementation Details	14-18
9.	Results & Findings	19-20
10.	Security Recommendations	21
11.	Learning Outcomes	22
12.	Conclusion	23
13.	Future Enhancements	24
14.	References	25

List of Figures

Figure No.	Description	Page No.
Figure 7.1	Workflow of the Linux Privilege Escalation Automation Toolkit	12
Figure 8.1	System Information Collection Output	13
Figure 8.2	Toolkit Execution Output	14
Figure 8.3	Enumeration of SUID Binaries	15
Figure 8.4	Cron Job Enumeration Output	15
Figure 8.5	Sudo Privilege Enumeration	16
Figure 8.6	Kernel Version Detection	17

1. ABSTRACT

Linux operating systems are extensively used in enterprise servers, cloud infrastructures, and security-critical environments due to their robustness, flexibility, and open-source nature. Despite built-in security mechanisms, Linux systems often face privilege escalation vulnerabilities caused by misconfigured permissions, insecure services, excessive user privileges, and outdated kernel versions. Privilege escalation occurs when a low-privileged user gains unauthorized root-level access, potentially compromising the entire system.

This internship project presents the design and implementation of a **Linux Privilege Escalation Automation Toolkit** that automates the detection of potential privilege escalation vectors. The toolkit performs safe auditing without executing exploitation code, scanning for **SUID/SGID binaries, weak file and directory permissions, misconfigured cron jobs, insecure sudo rules, vulnerable services, and outdated kernels**.

The toolkit was developed using **Python and Bash scripting** on Kali Linux within a VMware virtual environment, ensuring a safe and isolated testing setup. The final output of the toolkit includes **structured security reports, vulnerability tables, screenshots of system scans, and mitigation recommendations**, enabling both learning and real-world system hardening.

The toolkit is designed to safely identify potential privilege escalation vulnerabilities without performing exploitation, making it suitable for educational purposes, security audits, and practical training in system hardening. By identifying misconfigurations and weaknesses in Linux systems, the project helps improve overall system security and provides actionable insights for administrators and security professionals.

The project bridges theoretical cybersecurity concepts with practical real-world implementation, providing hands-on experience with **red-team enumeration techniques** and **blue-team defensive auditing practices**. Overall, this internship project provides a comprehensive understanding of Linux privilege escalation, strengthens practical cybersecurity skills, and equips students with the knowledge to perform automated system security auditing in real-world scenarios.

2. INTRODUCTION

Linux has become the backbone of modern computing infrastructure, powering web servers, cloud platforms, containers, Internet of Things (IoT) devices, and large-scale enterprise systems. Due to its open-source nature, flexibility, stability, and strong community support, Linux is widely adopted across industries for mission-critical applications. Organizations rely on Linux for hosting applications, managing databases, and running security-sensitive workloads.

Although Linux provides advanced built-in security mechanisms such as file permissions, user and group management, access control, and process isolation, system security heavily depends on correct configuration and regular auditing. Improper configurations, excessive permissions, and lack of security awareness often result in critical security gaps. These gaps can be exploited by attackers to compromise system integrity and confidentiality.

One of the most serious security threats in Linux environments is **privilege escalation**. Privilege escalation occurs when a user with limited or normal access exploits vulnerabilities or misconfigurations in the system to gain elevated privileges such as root or administrator access. Once root access is obtained, an attacker gains complete control over the system. This allows malicious activities such as modifying system configuration files, installing persistent backdoors, disabling security mechanisms, creating unauthorized users, and accessing or exfiltrating sensitive data.

Privilege escalation vulnerabilities commonly arise due to several misconfigurations and weaknesses present in Linux systems. Some of the most frequent causes include:

- **Misconfigured SUID/SGID binaries**, which allow programs to execute with elevated privileges and can be abused if insecure binaries are present
- **World-writable files and directories**, enabling unauthorized modification of files executed by privileged users or services
- **Insecure cron jobs running with root privileges**, where writable scripts are executed automatically by the system
- **Incorrect sudo configurations**, such as NOPASSWD rules that allow users to execute privileged commands without authentication

- **Vulnerable or outdated Linux kernels**, which may contain known privilege escalation exploits

Traditionally, identifying such vulnerabilities requires manual auditing of system configurations, permissions, and services. Manual security auditing demands deep technical expertise, significant time investment, and careful analysis. In large or complex environments, this approach becomes inefficient, inconsistent, and prone to human error. As a result, many privilege escalation vulnerabilities remain undetected until they are actively exploited.

Automation plays a crucial role in overcoming these challenges. Automated security auditing enables consistent, repeatable, and efficient analysis of system configurations and permissions. By automating privilege escalation checks, security professionals can quickly identify potential attack vectors and reduce the risk of system compromise.

This project aims to develop a **Linux Privilege Escalation Automation Toolkit** that systematically scans Linux systems for common privilege escalation risks. The toolkit is designed to replicate real-world penetration testing enumeration techniques used by attackers during the reconnaissance phase, while strictly avoiding exploitation. This ensures that the tool remains safe for educational, defensive, and auditing purposes.

Through this project, students gain practical exposure to real-world cybersecurity concepts, including red-team enumeration techniques and blue-team defensive strategies. The automated toolkit helps bridge the gap between theoretical knowledge and practical implementation, enabling better understanding of Linux security hardening and system auditing practices.

3. PROBLEM STATEMENT

Linux systems often contain hidden privilege escalation vulnerabilities due to poor system administration practices, legacy configurations, and the lack of regular security audits. Over time, misconfigured permissions, insecure services, and outdated system components accumulate, creating potential attack vectors that remain unnoticed by administrators.

Manual identification of privilege escalation vulnerabilities is a complex and time-consuming process. It requires deep technical expertise, careful inspection of system files, permissions, services, and kernel versions, and is highly prone to human error. In large or production environments, manual auditing becomes inefficient and inconsistent, increasing the risk of undetected vulnerabilities.

Organizations commonly face several challenges while assessing Linux system security, including:

- Difficulty in identifying **exploitable SUID/SGID binaries** scattered across the filesystem
- Lack of visibility into **world-writable system files and cron jobs** executed with root privileges
- **Insecure sudo configurations** that grant excessive or unrestricted administrative access
- **Outdated or vulnerable Linux kernel versions** susceptible to known privilege escalation exploits

Due to these challenges, many privilege escalation risks remain unaddressed, leaving systems vulnerable to compromise. There is a strong need for an **automated, safe, and educational security auditing tool** that can systematically identify potential privilege escalation vectors without performing exploitation. Such a tool can assist students, system administrators, and security professionals in understanding, detecting, and mitigating privilege escalation vulnerabilities in Linux environments.

4. PROJECT OBJECTIVES

The primary objectives of this internship project are:

1. **Development of an Automated Toolkit:-** To design and implement a comprehensive automation tool capable of identifying Linux privilege escalation vulnerabilities efficiently, including SUID/SGID binaries, misconfigured services, cron jobs, sudo rules, and kernel weaknesses.

2. **Practical Exposure to Security Auditing:-** To provide students with hands-on experience in real-world security auditing techniques, bridging theoretical knowledge with practical application in controlled and safe environments.

3. **Understanding Attacker Techniques:-** To study and replicate the methodologies used by attackers for system enumeration, privilege escalation, and vulnerability discovery, in order to enhance defensive and preventive skills.

4. **Defensive and Mitigation Skills:-** To assist system administrators and blue-team professionals in identifying misconfigurations and implementing effective mitigation strategies to harden Linux systems against privilege escalation.

5. **Professional Reporting:-** To generate structured, clear, and actionable security assessment reports that categorize findings by severity and provide mitigation recommendations, simulating professional cybersecurity documentation.

6. **Safe, Detection-Only Execution:-** To ensure that the toolkit performs only **safe detection and analysis**, without executing any live exploits, making it suitable for educational and audit purposes while maintaining system integrity.

7. **Skill Development: -** To enhance students' practical skills in Linux system security, scripting, automated auditing, and reporting, preparing them for real-world cybersecurity roles and assessments.

5. SCOPE OF THE PROJECT

The scope of this internship project defines the functional boundaries of the Linux Privilege Escalation Automation Toolkit. The project focuses on **safe detection and analysis** of privilege escalation vulnerabilities without performing exploitation. The scope is categorized into **In-Scope** and **Out-of-Scope** activities to clearly differentiate implemented functionalities from restricted operations.

5.1 In-Scope

The following activities are included within the scope of the project:

- Detection and enumeration of **SUID/SGID binaries** across the Linux filesystem that may allow privilege escalation if misconfigured.
- Identification of **weak file and directory permissions**, including world-writable files that can be abused by unprivileged users.
- Enumeration and analysis of **cron jobs executed with root privileges**, with emphasis on identifying writable scripts and insecure configurations.
- Detection of **insecure sudo configurations**, such as excessive privileges and NOPASSWD rules that may enable unauthorized command execution.
- **Kernel version analysis** to identify outdated or potentially vulnerable Linux kernels and provide vulnerability awareness.
- Generation of a **structured security assessment report** containing identified vulnerabilities, risk levels, and recommended mitigation steps.

5.2 Out-of-Scope

The following activities are explicitly excluded from the scope of the project to ensure ethical and safe execution:

- Exploiting identified vulnerabilities to gain elevated privileges.
- Executing kernel-level exploits or proof-of-concept attack code.
- Gaining unauthorized root or administrative access to the system.
- Modifying, deleting, or altering system files and configurations.

6. TOOLS & TECHNOLOGIES USED

1. **Operating System**
    - Kali Linux installed on a **VMware virtual machine** to provide an isolated and safe testing environment.
2. **Programming & Scripting Languages**
    - Python: for scripting automated scanning modules.
    - Bash: for Linux command automation and integration of system-level checks.
3. **Linux Commands & Utilities**
    - **find:** locate SUID/SGID binaries and search files.
    - **ls -la:** recursive directory listing for permission analysis.
    - **sudo -l:** analyze sudo privilege configurations.
    - **systemctl:** investigate systemd services and configurations.
    - **crontab:** enumerate cron jobs executed by root or other users.
    - **uname -a:** obtain kernel version and system information.
    - **grep, awk, sed:** text parsing, filtering, and report formatting.
4. **Documentation & Visualization Tools**
    - MS Word / Google Docs: project documentation.
    - Draw.io: creation of flowcharts and architecture diagrams.
    - Microsoft PowerPoint: preparation of final presentation.

All tools were selected to ensure **safe, automated, and reproducible testing**, providing practical experience for students in Linux system security auditing.

7. METHODOLOGY / WORKFLOW

The Linux Privilege Escalation Automation Toolkit follows a **modular, systematic, and structured workflow** to ensure accurate detection of privilege escalation vulnerabilities. The methodology is designed to simulate **real world security auditing and penetration testing enumeration techniques** while maintaining a safe, detection-only approach.

The overall workflow is divided into four major stages, each responsible for a specific task in the security assessment process.

Step 1: System Information Collection

In this initial phase, the toolkit gathers essential system-level information to understand the execution environment. This information helps determine whether the user has limited privileges and assists in identifying kernel-related risks.

The following details are collected:

- Identification of the **current logged-in user** and associated group memberships
- Detection of the **Linux distribution and operating system version**
- Capturing the **kernel version and system architecture**
- Verification of the **current privilege level** (root or non-root user)

Step 2: Privilege Escalation Vector Scanning

In this stage, the toolkit performs automated enumeration to identify common privilege escalation vectors present in Linux systems. Multiple scanning modules are executed to analyze different attack surfaces.

The toolkit performs the following checks:

- Scans the filesystem to identify **SUID/SGID binaries** that may allow privilege escalation if misconfigured
- Detects **world-writable files and directories** that can be modified by unprivileged users

- Enumerates **cron jobs running with root privileges** and checks for writable scripts
- Analyzes **sudo configurations** to identify excessive privileges or NOPASSWD rules
- Examines **systemd services** running as root and checks for insecure configurations

Step 3: Analysis Engine

After data collection, the toolkit processes and analyzes the results to identify meaningful privilege escalation risks. This step reduces noise and improves accuracy.

The analysis engine performs:

- Matching of identified findings with **predefined vulnerability patterns**
- Identification of **high-risk privilege escalation paths**
- Filtering of **false positives** to ensure reliable reporting
- Categorization of vulnerabilities based on **severity level**

This phase helps convert raw system data into actionable security insights.

Step 4: Report Generation

In the final stage, the toolkit generates a **structured and professional security report** summarizing all identified issues. The report is designed to be easily understood by system administrators and security teams.

The report includes:

- A categorized list of detected vulnerabilities
- Severity assessment for each finding
- Description of exploitation possibility
- Recommended **mitigation and remediation steps**

Workflow Diagram

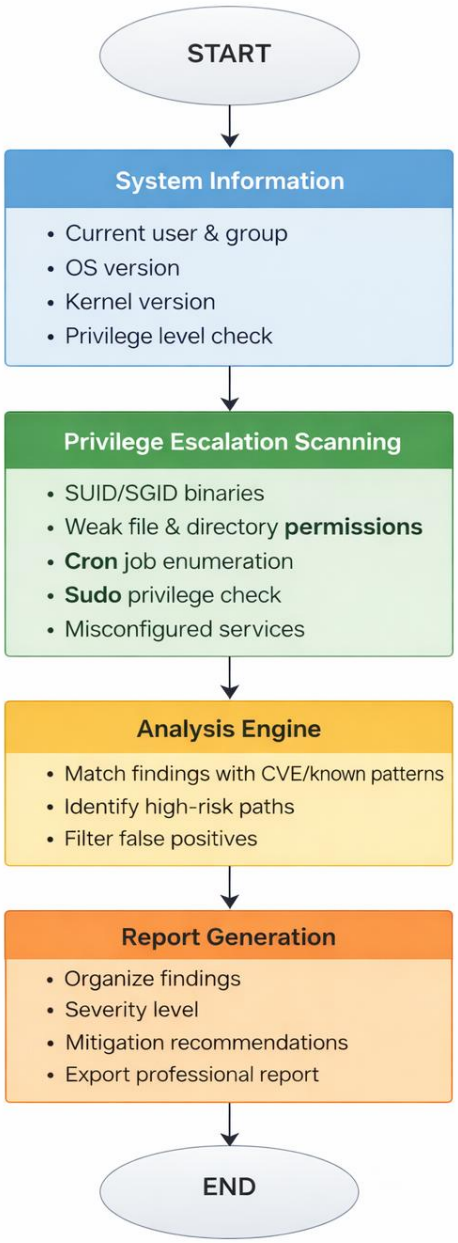

Figure 6.1: Workflow of the Linux Privilege Escalation Automation Toolkit

The figure illustrates the step-by-step workflow of the toolkit, starting from system information collection to final security report generation. Each module operates independently to ensure modularity and safe detection of privilege escalation vulne-

8. IMPLEMENTATION DETAILS

The Linux Privilege Escalation Automation Toolkit was implemented and tested in a controlled and isolated virtual environment to ensure safe execution. The toolkit was executed using a **non-root user account** to accurately simulate real world attacker enumeration and privilege escalation discovery techniques.

8.1 Execution Environment

- Operating System: Kali Linux
- Virtualization Platform: VMware
- User Privilege Level: Non-root (standard user)
- Scripting Languages: Python and Bash

The use of a virtual machine ensures that all scanning activities remain isolated from the host system and do not impact production environments.

Figure 8.1: System Information Collection

This screenshot shows the collection of basic system information including current user, group membership, operating system details, and kernel version.

8.2 Toolkit Execution

The toolkit is executed from the terminal using a single command, which sequentially triggers all enumeration modules. During execution, the toolkit collects system information, scans for privilege escalation vectors, and stores findings for further analysis.

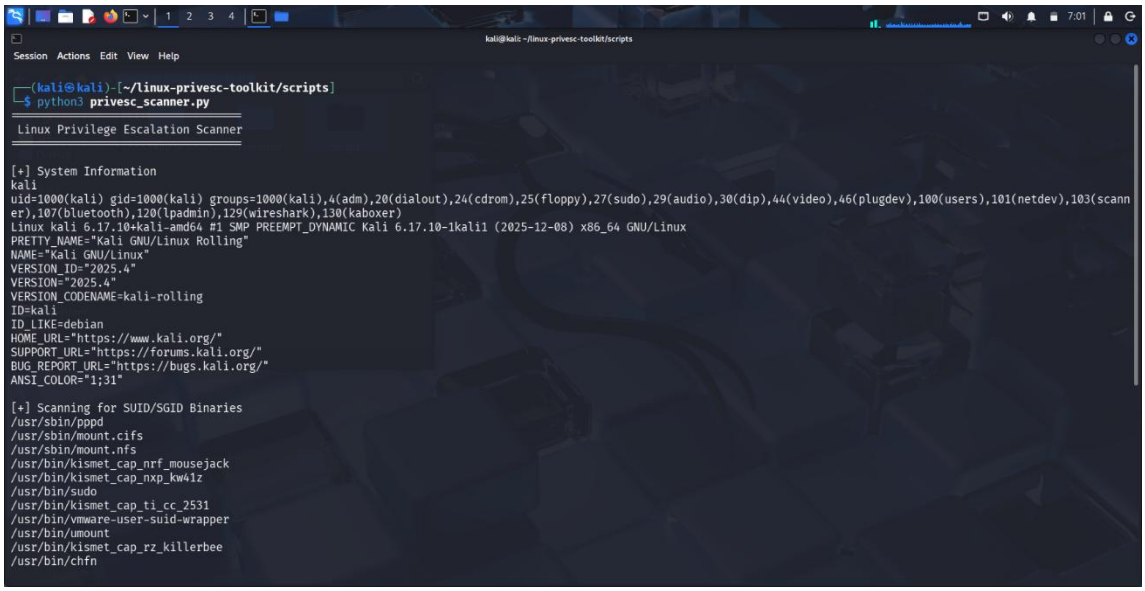

Figure 8.2: Toolkit Execution Output

This screenshot demonstrates the execution of the Linux Privilege Escalation Automation Toolkit, which initiates multiple enumeration modules to identify potential privilege escalation vectors.

8.3 SUID/SGID Binary Discovery

In this module, the toolkit scans the filesystem to identify binaries with SUID and SGID permissions. Such binaries can allow users to execute commands with elevated privileges if misconfigured.

The discovered binaries are compared against known high-risk binaries to highlight potential exploitation paths.

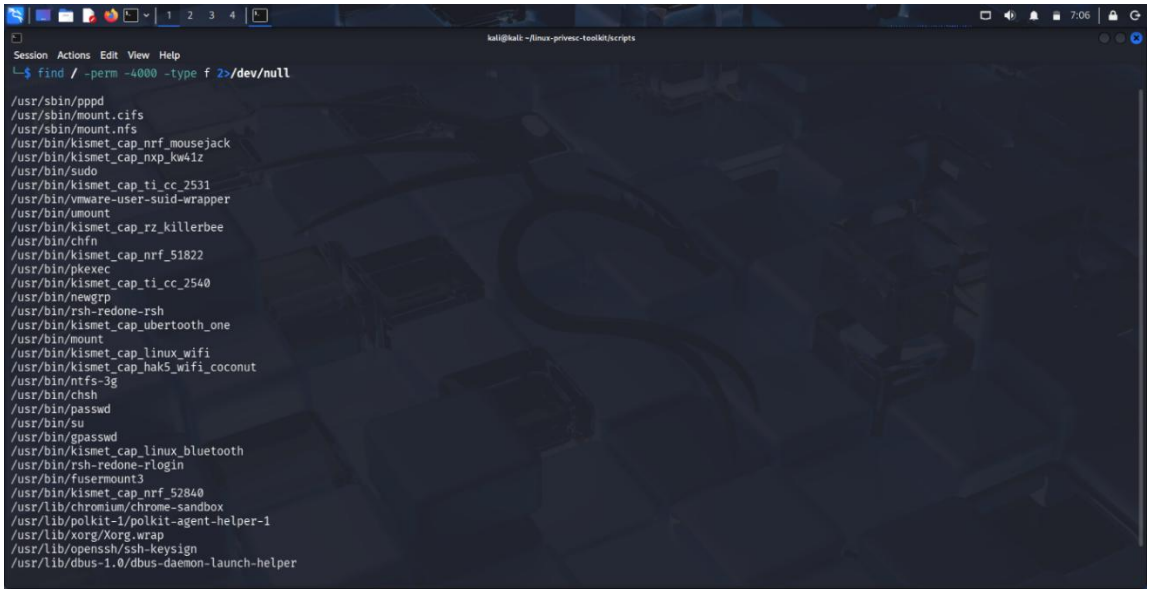

Figure 8.3: Enumeration of SUID Binaries

This screenshot displays binaries with SUID permissions. Misconfigured SUID binaries can allow unprivileged users to execute commands with elevated privileges, leading to potential privilege escalation.

8.4 Cron Job Enumeration

This module enumerates system-level and user-level cron jobs, focusing on tasks executed with root privileges. Writable scripts or improperly secured cron jobs are flagged as potential privilege escalation vectors.

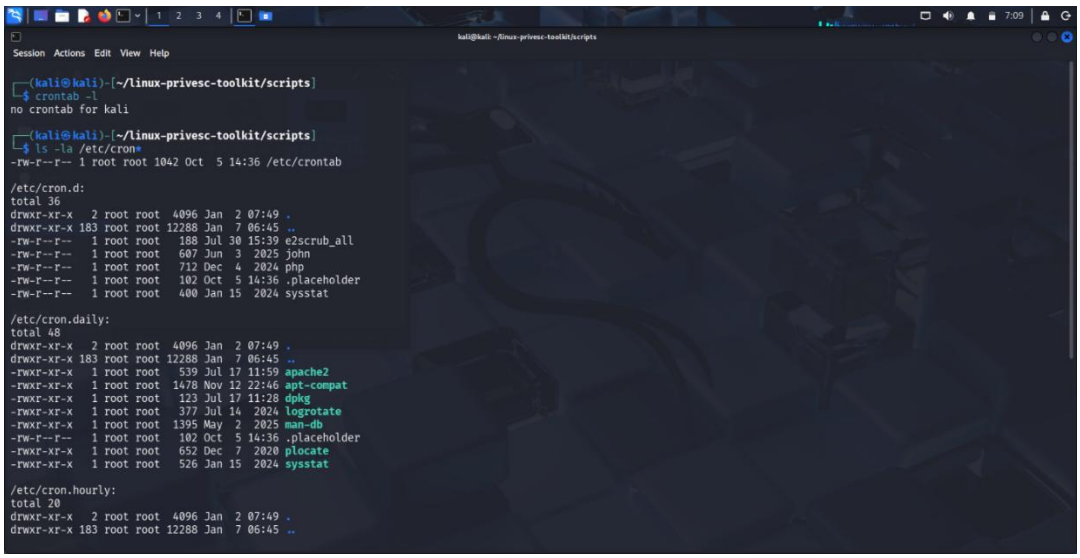

Figure 8.4: Cron Job Enumeration Output

This screenshot shows scheduled cron jobs and cron directories. Writable scripts executed by root cron jobs may allow attackers to execute malicious commands with elevated privileges.

8.5 Sudo Privilege Analysis

The toolkit uses sudo privilege enumeration to identify commands that can be executed with elevated privileges without authentication. Misconfigured sudo rules are marked as high-risk findings.

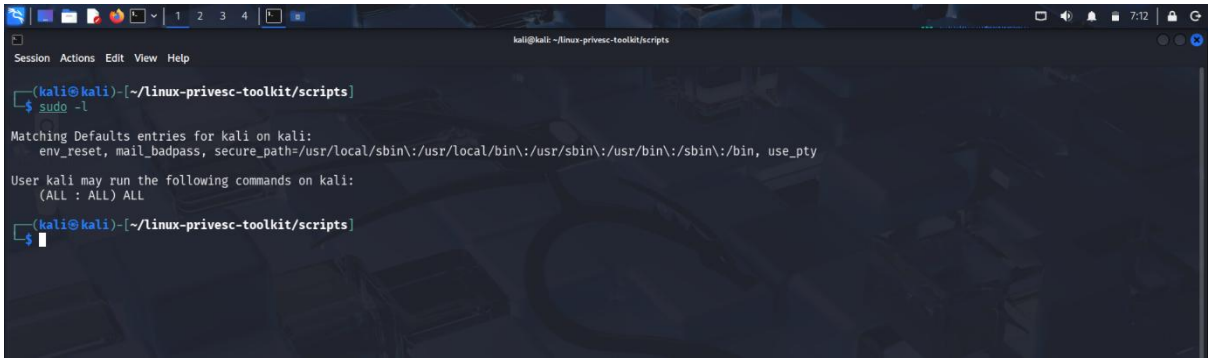

Figure 8.5: Sudo Privilege Enumeration

This screenshot displays sudo permissions assigned to the current user. Insecure sudo configurations such as NOPASSWD rules may lead to unauthorized privilege escalation.

8.6 Kernel Version Detection

The kernel version is detected and analyzed to identify outdated or vulnerable kernels. The toolkit highlights potential security risks while avoiding execution of any exploit code.

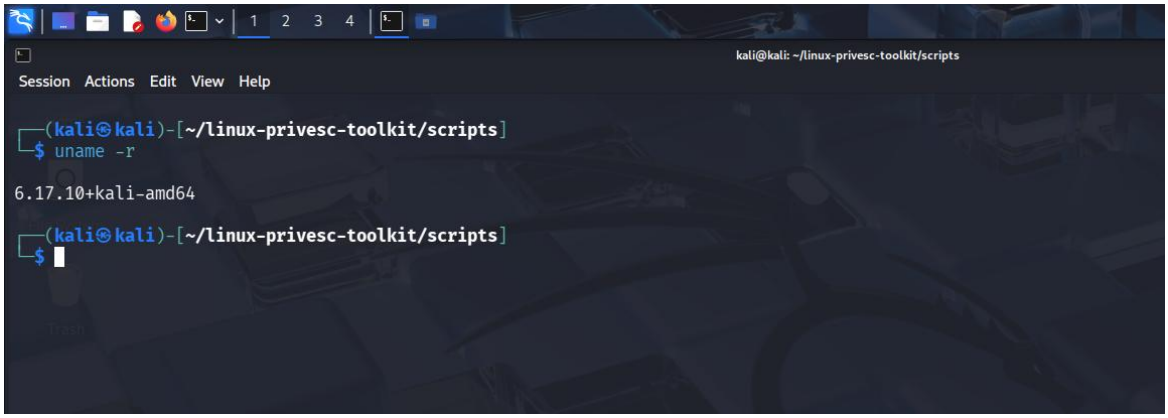

Figure 8.6: Kernel Version Detection

This screenshot shows the running Linux kernel version. Outdated kernels may contain known vulnerabilities that can be exploited for privilege escalation.

8.7 Documentation of Findings

Each identified vulnerability is documented with:

- Description of the issue
- Potential security impact
- Severity level
- Recommended mitigation steps

This structured documentation ensures that the results are understandable for system administrators and security professionals.

9. RESULTS & FINDINGS

This section presents the results obtained after executing the Linux Privilege Escalation Automation Toolkit. The findings highlight various privilege escalation risks identified during automated enumeration. Each vulnerability is categorized based on its type, severity level, and potential impact on system security.

9.1 Summary of Identified Vulnerabilities

Vulnerability Type	Risk Level	Description
SUID Binaries	High	Presence of misconfigured SUID binaries that may allow privilege escalation
Cron Jobs	High	Writable cron scripts executed with root privileges
Sudo Rules	Medium	Insecure sudo configurations such as NOPASSWD rules
Kernel Version	Medium	Outdated Linux kernel detected with known vulnerabilities

9.2 SUID/SGID Binary Findings

The toolkit identified multiple binaries with SUID/SGID permissions. Some of these binaries are considered high risk as they can potentially be abused by unprivileged users to execute commands with elevated privileges.

Risk Level: High
Reason: Misconfigured SUID binaries can directly lead to full root compromise if exploited.

9.3 Cron Job Vulnerabilities

Several cron jobs running with root privileges were identified. In some cases, the scripts executed by these cron jobs were found to have weak file permissions, allowing modification by non-root users.

Risk Level: High
Reason: Writable cron scripts can be replaced or modified to execute malicious commands as root.

9.4 Sudo Privilege Misconfigurations

The toolkit detected sudo rules that allow certain commands to be executed without requiring a password. While not always dangerous, such configurations can be misused if not carefully restricted.

Risk Level: Medium
Reason: Excessive sudo privileges increase the attack surface for privilege escalation.

9.5 Kernel Version Analysis

The system kernel version was analyzed and found to be outdated. Older kernels may contain known vulnerabilities that could be exploited for privilege escalation.

Risk Level: Medium
Reason: Unpatched kernels are susceptible to publicly known exploits.

9.6 Overall Risk Assessment

The results indicate that **misconfigurations at multiple levels** significantly increase the risk of privilege escalation. High-risk findings such as SUID binaries and writable cron jobs pose immediate threats and require urgent remediation.

10. SECURITY RECOMMENDATIONS

Based on the findings from the Linux Privilege Escalation Automation Toolkit, the following recommendations are proposed to mitigate identified vulnerabilities and enhance overall system security. These recommendations are designed to be **practical, actionable, and safe** for real-world implementation.

1. **Remove Unnecessary SUID/SGID Permissions**:- Any binaries that do not require elevated privileges should have SUID/SGID bits removed. This minimizes the risk of privilege escalation through misconfigured executables.
2. **Restrict File and Directory Permissions to Least Privilege**:- Ensure that files and directories are writable only by authorized users. World-writable files should be avoided or restricted to prevent unauthorized modifications.
3. **Secure Cron Jobs and Scripts**:- Cron jobs executed as root should have restricted permissions and be reviewed periodically. Writable scripts in cron jobs should be corrected or removed to prevent malicious replacement.
4. **Review and Harden Sudo Rules Regularly**:- Sudo configurations should be audited to remove excessive privileges and NOPASSWD rules wherever possible. Only required commands should be permitted for each user.
5. **Patch and Update Linux Kernel**:- The system kernel should be updated regularly to mitigate known vulnerabilities. Outdated kernels can be exploited to escalate privileges and compromise system integrity.
6. **Perform Periodic Automated Audits**:- Regular execution of the automated toolkit or similar auditing tools ensures that new misconfigurations or vulnerabilities are detected promptly, reducing the risk of privilege escalation over time.

11. LEARNING OUTCOMES

This internship project provided valuable hands-on exposure to Linux system security and privilege escalation assessment. Through the design and implementation of the Linux Privilege Escalation Automation Toolkit, the following learning outcomes were achieved:

- **Practical Understanding of Linux Privilege Escalation**:- Gained in-depth knowledge of how privilege escalation vulnerabilities arise due to misconfigurations in Linux systems and how attackers identify such weaknesses in real environments.

- **Hands-on Experience with Automated Security Auditing**:- Learned how to automate security enumeration processes using scripting, enabling faster, consistent, and repeatable vulnerability detection compared to manual auditing.

- **Improved Scripting and System Analysis Skills**:- Enhanced proficiency in Python and Bash scripting while integrating Linux system commands to analyze permissions, services, cron jobs, and kernel information.

- **Exposure to Real World Red Team and Blue Team Practices**:- Developed an understanding of red team enumeration techniques used in penetration testing as well as blue team defensive strategies for auditing and mitigation.

- **Understanding of Secure and Ethical Testing Practices**:- Learned the importance of conducting security assessments in a safe, detection-only manner without exploiting vulnerabilities or impacting system integrity.

12. CONCLUSION

This internship project successfully demonstrates the design, development, and implementation of a **Linux Privilege Escalation Automation Toolkit** that safely identifies security misconfigurations across the system. The toolkit automates the detection of **SUID/SGID binaries, insecure sudo rules, writable cron jobs, and outdated kernel versions**, providing a safe and structured approach to privilege escalation auditing without executing harmful exploits.

Through this project, students gained **hands-on experience with Linux system enumeration, file and directory permissions analysis, cron job auditing, and kernel version checks**, significantly enhancing their practical cybersecurity skills. The use of **Python and Bash scripting** to automate security checks provided valuable insights into scripting, automation, and reporting techniques used by professional penetration testers and system auditors.

The toolkit's findings highlighted **high-risk vulnerabilities**, and the structured report provided **actionable mitigation steps**, emphasizing the importance of **periodic security audits** and proactive system hardening. This reflects a real-world approach to balancing red team enumeration techniques with blue team defensive strategies.

Overall, this project not only strengthened technical and analytical skills but also improved **documentation, report writing, and professional presentation abilities**. The internship experience has prepared students for practical roles in cybersecurity, providing a foundation for future learning, ethical hacking practice, and continuous system security improvement.

13. FUTURE ENHANCEMENTS

The Linux Privilege Escalation Automation Toolkit can be further enhanced in the future to increase its functionality, usability, and real-world applicability. The following enhancements are proposed:

1. **Integration with CVE Databases**:- Linking the toolkit with publicly available CVE (Common Vulnerabilities and Exposures) databases will allow real-time identification of known vulnerabilities in the system, especially kernel and service exploits. This will provide a **more comprehensive vulnerability assessment**.
2. **Web-Based Reporting Dashboard**:- Developing a web interface for the toolkit will enable **interactive visualization of findings**, allowing system administrators and security teams to easily interpret results and track vulnerabilities over time.
3. **Real-Time Alert Generation**:- Future versions can incorporate **real-time notifications** for newly detected high-risk misconfigurations, helping administrators quickly respond to potential security threats and maintain continuous monitoring.
4. **Support for Multiple Linux Distributions**:- Expanding compatibility beyond Kali Linux to other distributions such as Ubuntu, CentOS, and Debian will make the toolkit **more versatile and widely usable** in diverse enterprise environments.
5. **Modular Expansion for Custom Checks**:- Allow users to add custom scanning modules to detect organization-specific misconfigurations or emerging security issues, increasing the **flexibility and adaptability** of the toolkit.

14. REFERENCES

- **Linux Man Pages** – The manual pages for Linux commands and utilities. Available at: https://man7.org/linux/man-pages/
- **GTFOBins** – A curated list of Unix binaries that can be used by an attacker to bypass local security restrictions. Available at: https://gtfobins.github.io/
- **MITRE CVE Database** – Common Vulnerabilities and Exposures database for tracking publicly known cybersecurity vulnerabilities. Available at: https://cve.mitre.org/
- **Kali Linux Documentation** – Official documentation for Kali Linux tools and system usage. Available at: https://www.kali.org/docs/
- **OWASP – Security Best Practices** – Open Web Application Security Project guidance for secure system configurations. Available at: https://owasp.org/